

Fundamentals of NLP

NLP basics relevant to FYP

Daniel Rice

1 Prerequisite NLP Information

This section briefly defines and explains some of the fundamentals in NLP, which are referenced in the NLP-research section of this report's corresponding final-year- project document.

1.1 Tokenisation

Tokens are produced as a result of tokenisation, which is a process that breaks the input data – text, for NLP – into smaller units: tokens. There are different methods of tokenisation, for example breaking a sequence of text, e.g. a sentence, down to each word in the sentence (word tokenisation), an example of an array storing word-tokens is ["the", "fox", "jumped", "over", "the", "lazy", "dog"]. However, the method of tokenisation depends on the context and problem statement, for example it may be more appropriate to tokenise by subword, or characters in a (sub)word [7]. Note, however, that whilst here and throughout this document, tokenisation examples are shown as they relate/apply to NLP, however tokenisation for computer vision, for example, may be a frame being chunked, each smaller chunk then being a token. Tokenisation is used in NLP, since it allows for example a full-sentence to be broken down to individual components (words), which allows each word to be processed separately (in word-tokenisation), allowing for lexical analysis, such as the meaning of each word in context, NER, sentiment analysis, among others [14], examples of sentiment analyses for different NLP techniques are given in the FYP report, for example BoW in section BoW_example of FYP.

1.2 Embedding

Embedding is the process by which tokens are mapped to points in N -dimensional space, the position then stored in a vector, per token. There are two types of resultant vectors produced by an embedding process (of which there are many, some of which are covered in literature_existing_nlp_techniques of the FYP main report) sparse embeddings, and dense embeddings [3].

1.2.1 Sparse-Embedding Matrix

A sparse-embedding matrix is a matrix containing sparsely-embedded vectors for a given corpus, vectors embedded using a sparse method, such as one-hot encoding (such as BoW uses; see para:bag_of_words of the FYP main report). When using a sparse-embedding method, such as one-hot encoding, the number of dimensions will be the number of unique tokens (words, for NLP) in the corpus. With reference to the example showed in the BoW explanation in the FYP main report, in a given token's sparsely-embedded vector the majority of elements are zero, with a "1" element denoting the link between this unique-token and its corresponding position in the built lexicon, e.g. for a sparse-vector with contents [0,0,1] representing the token "cat" , "cat"'s index in the arrayed-lexicon is 2 (zero-indexed). Since the number of dimensions are equal to the number of unique tokens, sparse embeddings are inherently very-high dimensionality, resulting in issues explained in [section 1.3.1](#).

Use Cases

The structure of sparse-embeddings (large number of zeroed values) can be useful for keyword-lookup or matching, since the zero-values indicate an absence in their corresponding tokens, given the embedded-input [3], which can be coupled with vector indexing to map vectors to their nearest-neighbours. This enables keyword-lookup for an inputted keyword, allowing similar present keywords [in the input-corpus] to be returned, resulting in fast similarity

lookups [15], but only when a method of embedding involves associating a token with its nearest-neighbour, such as SPLADE.

SPLADE

SPLADE uses modern NLP techniques, such as transformers (see `subsubsec:transformers` of the FYP main report), to generate a sparse-embedding vector of the input, where the transformer aspect captures semantically-similarities between the input-tokens (via self-attention: `lit:self-attention`), this can then be coupled with inverted indexing to map a specific keyword (-to-be looked-up) to corpora containing that specific keyword, or contextually-similar keywords, which may, for example, associate “flights” with “airlines”, both pertaining to air-travel [8], this can hence be effective when used in search engines’ back-end, for example, or other information-retrieval systems.

1.2.2 Dense-Embedding Matrix

Matrix-collation of densely-embedded vectors. Densely-embedded vectors are, hence the name, more compact than sparsely-embedded vectors, containing few (or no) zeroed-values. Crucially, however, the embedding process typically involves a neural-network approach (e.g. Word2Vec; BERT (see FYP main report corresponding sections for each Word2Vec and BERT)), allowing semantic-relationships to be captured in the resultant, ordered, embedding. Given the few-, or non-, zeroed-values nature of **dense** embeddings, these are inherently lower in dimensionality than sparse embeddings. The issues arising from the curse of dimensionality (section 1.3.1) do not apply here, given the lower dimensionality, as such they are faster to compute than sparse embeddings¹.

In NLP, dense embeddings are used to (significantly-better) capture semantic-meaning from the input. Therefore, unlike sparse-embeddings, these can be used for strong(er) text analysis, for example the sentiment of a review (customer’s perceived satisfaction) can be contained within its dense embedding (unlike a bag of words approach with a sparse-embedding, particularly given BoW models/sparse-embedding vectors are unordered), this then an input to a downstream task, such as classification, wherein the review would be classified into categories, e.g. binary classification of good/bad review. With this, the dense embedding is learnt through a technique such as masked-language modelling (explained in the FYP main report).

1.3 Dimensionality

Dimensionality, informally, refers to the number of coordinates to define a point in a given- N - dimensional space [12]. With embedding vectors, this is the length of the vector (number of elements in the vector).

Visualising embeddings in a two/three-dimensional space can be achieved by using a dimensionality-reduction technique such as Principal-Component Analysis (PCA), retaining the most important data-points, whilst reducing the number of features (aspects) of the embedding (see below section 1.3.1).

1.3.1 Curse of Dimensionality

The curse of dimensionality, introduced by Bellman, refers to problems that arise in data with high dimensionality, for example a sharp decrease in the accuracy of a model (possibly due to exploding/vanishing gradient problem; see FYP section on this for explanation) when increasing the number of dimensions in the data, which also causes problems for data-acquisition itself, since the amount of data required increases exponentially with increases in dimensionality (due to high size of feature space). Further, computational expense of processing an increased amount of data, from an increase in features (which can, up to a point, potentially increase accuracy on downstream tasks, however – particularly when initially building-up lower-dimensional data).

Given that dimensionality is the amount of features/attributes comprising some data, e.g. aforementioned embedding vector, an example of requiring exponentially more data is if the vector has particularly-high dimensionality, per additional vector computed, there are a particularly-high number of fields to fill, i.e. if all vectors have 250 dimensions, each vector must have 250 data points (entries) (/length of vector is 250), each field’s value referring to its respective feature. Therefore, with high dimensionality, there is a need to have a large amount of data per vector, as such

¹It is worth noting that processing sparse embeddings tends to result in a RAM-bottleneck, as a result of the very-large number of (zeroed) elements; this is not the case for an equivalent-input dense embedding, as such these [dense embeddings] require less RAM bandwidth, compared with processing sparse embeddings, though dense-embeddings can be (and, actually, in practice, often are much) more computationally-expensive to map/create initially.

with dimension-increase, the performance required for accurate results by ML algorithms requires an exponential-increase in data points, since for a feature which can have 2 possible values (binary), assuming 20 data points are needed for the model to perform accurately, and there is 1 dimension, then the number of unique values is 2^1 , then applying this to the number of data points needed (20), $2^1 \cdot 20 = 40$, in general $numValsPerFeature^{numDimensions} \cdot numDataPointsNeeded$ (in this example, $2^k \cdot 20$, where k is the number of dimensions for these parameters (values per feature, and minimum data points required)). Therefore, adding another dimension results in $2^2 \cdot 20 = 80$, and once more with 3 dimensions $2^3 \cdot 20 = 160$, this showing the exponential increase (in this example) in the data required for this model to make accurate predictions [11].

Reducing Dimensionality via PCA

Given some data containing a large number of features, PCA transforms these into new features called principal (leading; “most important”) components, which are based on direction (eigenvector) and perceived importance (eigenvalues) of the features [6], calculating eigenvectors and corresponding eigenvalues from a covariance matrix (comparing how each feature varies with every other feature – but also including itself (diagonal elements in covariance matrix showing variance within the given feature itself – deviation of its points from mean point)).

Once the principal components have been calculated (eigenvectors of covariance matrix), each with importance (eigenvalues), find the eigenvectors with the highest eigenvalues (choosing the direction (eigenvector) with the greatest amount of variance, since variation of data indicates useful information, thereby capturing this when projected onto the original data). Projecting the eigenvectors over the original data (visualisation) shows the principal components, taking away the original data – reduced dimensionality to the chosen, most-relevant, principal components – thereby retaining the deemed most-useful information, whilst discarding all else. The more principal components that are chosen to be retained, the more the reduced data will resemble the original data (i.e. higher dimensionality, but more data, potentially harder to visualise, therefore) [6].

1.4 Backpropagation

Backpropagation is used in training neural networks (NNs) to minimise the loss function – the difference between model output and expected output, for some given training input data from the overall training data, this is calculated via a metric showing the error, for regression problems, this may be Mean Squared Error (MSE) (see eq. (1)), which measures the average squared difference between the actual value (in the training data) and the predicted value (from the network) [5] – a lower MSE therefore shows the network’s prediction is closer to the actual/expected value; greater prediction accuracy of the model [5]. MSE is useful as a loss function, given it is influenced by large errors, more so than small errors (due to squaring the error), focusing on the largest error produced by the network, rather than small errors. Calculating the gradients of each layer, backwards (from output layer to input layer), using the chain rule to calculate partial derivatives of each layer, with respect to each layer before (“before” referring to each layer closer to the output layer of the network), this enables determining how much a given adjustment in a given layer’s weights will impact the network’s overall output, and, by extension, how that impact the loss function – knowing how much to change a given weight is determined by gradient descent, see section 1.4.1.

This occurs by adjusting the weights of neurons by calculating the gradient per layer backwards (from output layer through the hidden layers, to the input layer) through the network; backward pass, see below. The amount to adjust each layer’s weights is determined by the calculated gradient, with respect to each previous layer (where previous layers are those closer to the final layer of the network, given this is the **backward** pass) [9].

$$\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2 \quad (1)$$

Showing MSE as a **cost** function (since this is calculated and summed over the whole training dataset, if this was calculating a single loss only, this would not include the summation aspect), where n is the number of data points in the training set², Y_i is the actual value of the i th data point, and $\hat{Y}_i$ is the predicted value of the i th data point [5].

²As you may have spotted, given this equation stated is a **cost** function, with very-large training datasets, n will be very large, as such if running this n times per epoch, this is very computationally expensive. One optimisation, therefore, is stochastic gradient descent, which uses only a (pseudo-)randomly- chosen subset of training data points, n is then the size of this subset of training data.

1.4.1 Gradient Descent

Given the gradients per layer have been calculated from backpropagation, gradient descent determines the amount to adjust the weights of a given layer in an attempt to reduce the overall loss on all training data (cost function) (until the point at which this cost function has been minimised, i.e. convergence has been reached, due to weights having reached global minimum in the given cost function).

The (inverse of the) gradient calculated by backpropagation determines the “direction to move in”, i.e. how much to increase or decrease a given weight – an additional scalar is also used to adjust a given weight, which is a hyperparameter known as the learning rate. If the learning rate value is high, the step size taken to adjust a given weight will be large, which presents issues when the network is close to convergence, given the weights may be adjusted by the amount of the learning rate around the minimum, so that the output “bounces” around the global minimum; if the learning rate is too low, the learning process will take a considerable amount of time (per epoch (/iteration of training, i.e. one epoch is complete after completing both forwards and backwards pass), little change is made, resulting in a higher number of epochs being required to converge), as such it is common to use a variable learning rate, which is higher to begin with, then gradually decays (this being a hyperparameter) [9].

References

- [1] Bernstein, D. J., Bhargavan, K., Bhasin, S., Chattopadhyay, A., Chia, T. K., Kannwischer, M. J., Kiefer, F., Paiva, T., Ravi, P. and Tamvada, G. [2024], 'KyberSlash: Exploiting secret-dependent division timings in kyber implementations', Cryptology ePrint Archive, Paper 2024/1049. [Online; accessed 29-September-2025]
Raw PDF link: <https://eprint.iacr.org/2024/1049.pdf>
In event of dead-link, or changed content, visit: <https://web.archive.org/web/20250929122737/https://eprint.iacr.org/2024/1049.pdf> (thanks to WayBack Machine).
URL: <https://eprint.iacr.org/2024/1049>
- [2] Ehren Kret, Rolfe Schmidt [2024], 'The pqxdh key agreement protocol', <https://signal.org/docs/specifications/pqxdh/>. [Online; accessed 03-September-2025]
In event of dead-link, or changed content, visit: <https://web.archive.org/web/20250903160207/https://signal.org/docs/specifications/pqxdh/> (thanks to WayBack Machine).
- [3] Frank Liu [2023], 'Sparse and dense embeddings', <https://zilliz.com/learn/sparse-and-dense-embeddings>. [Online; accessed 14-August-2025]
In event of dead-link, or changed content, visit: <https://web.archive.org/web/20250814160051/https://zilliz.com/learn/sparse-and-dense-embeddings> (thanks to WayBack Machine).
- [4] GeeksforGeeks contributors [2024], 'Implementation of diffie-hellman algorithm — GeeksforGeeks', <https://www.geeksforgeeks.org/computer-networks/implementation-diffie-hellman-algorithm/>. [Online; accessed 06-September-2025]
In event of dead-link, or changed content, visit: <https://web.archive.org/web/20250906130332/https://www.geeksforgeeks.org/computer-networks/implementation-diffie-hellman-algorithm/> (thanks to WayBack Machine).
- [5] GeeksforGeeks contributors [2025a], 'Mean squared error — GeeksforGeeks', <https://www.geeksforgeeks.org/maths/mean-squared-error/>. [Online; accessed 10-October-2025]
In event of dead-link, or changed content, visit: <https://web.archive.org/web/20251010150908/https://www.geeksforgeeks.org/maths/mean-squared-error/> (thanks to WayBack Machine).
- [6] GeeksforGeeks contributors [2025b], 'Principal component analysis(pca)', <https://www.geeksforgeeks.org/data-analysis/principal-component-analysis-pca/>. [Online; accessed 19-August-2025]
In event of dead-link, or changed content, visit: <https://web.archive.org/web/20250819142442/https://www.geeksforgeeks.org/data-analysis/principal-component-analysis-pca/> (thanks to WayBack Machine).
- [7] GeeksforGeeks contributors [2025c], 'What is tokenization in natural language processing (nlp)?', <https://www.geeksforgeeks.org/nlp/tokenization-in-natural-language-processing-nlp/>. [Online; accessed 13-August-2025]
In event of dead-link, or changed content, visit: <https://web.archive.org/web/20250813150946/https://www.geeksforgeeks.org/nlp/tokenization-in-natural-language-processing-nlp/> (thanks to WayBack Machine).
- [8] James Briggs [2023], 'Splade for sparse vector search explained', <https://www.pinecone.io/learn/splade/>. [Online; accessed 15-August-2025]
In event of dead-link, or changed content, visit: <https://web.archive.org/web/20250815153647/https://www.pinecone.io/learn/splade/> (thanks to WayBack Machine).
- [9] Mbali Kalirane [2025], 'Gradient descent vs. backpropagation: Whats the difference?', <https://www.analyticsvidhya.com/blog/2023/01/gradient-descent-vs-backpropagation-whats-the-difference/>. [Online; accessed 10-October-2025]
Note, on the date for this entry, the website claims the article was last updated on 14, May, 2025; the URL, however, indicates the page was first written in 2023. Since the version I am using in the report is the 14, May, 2025 version, 2025 is the year set in this BibTeX entry.
In event of dead-link, or changed content, visit: <https://web.archive.org/web/20251010143527/https://www.analyticsvidhya.com/blog/2023/01/gradient-descent-vs-backpropagation-whats-the-difference/> (thanks to WayBack Machine).

- [10] Moxie Marlinspike, Trevor Perrin [2016], 'The double ratchet algorithm', <https://signal.org/docs/specifications/doubleratchet/>. [Online; accessed 04-September-2025]
In event of dead-link, or changed content, visit: <https://web.archive.org/web/20250904170654/https://signal.org/docs/specifications/doubleratchet/> (thanks to WayBack Machine).
- [11] Shashmi Karanam [2021], 'Curse of dimensionality a curse to machine learning', <https://towardsdatascience.com/curse-of-dimensionality-a-curse-to-machine-learning-c122ee33bfeb/>. [Online; accessed 19-August-2025]
In event of dead-link, or changed content, visit: <https://web.archive.org/web/20250819173032/https://towardsdatascience.com/curse-of-dimensionality-a-curse-to-machine-learning-c122ee33bfeb/> (thanks to WayBack Machine).
- [12] Wikipedia contributors [2025a], 'Dimension — Wikipedia, the free encyclopedia', <https://en.wikipedia.org/w/index.php?title=Dimension&oldid=1305047132>. [Online; accessed 18-August-2025]
In event of dead-link, or changed content, visit: <https://web.archive.org/web/20250818172556/https://en.wikipedia.org/w/index.php?title=Dimension&oldid=1305047132> (thanks to WayBack Machine).
- [13] Wikipedia contributors [2025b], 'Kyber — Wikipedia, the free encyclopedia', <https://en.wikipedia.org/w/index.php?title=Kyber&oldid=1308538050>. [Online; accessed 05-September-2025]
In event of dead-link, or changed content, visit: <https://web.archive.org/web/20250905210520/https://en.wikipedia.org/w/index.php?title=Kyber&oldid=1308538050> (thanks to WayBack Machine).
- [14] Wikipedia contributors [2025c], 'Natural language processing — Wikipedia, the free encyclopedia', https://en.wikipedia.org/w/index.php?title=Natural_language_processing&oldid=1301380737. [Online; accessed 21-July-2025]
In event of dead-link, or changed content, visit: https://web.archive.org/web/20250723135411/https://en.wikipedia.org/w/index.php?title=Natural_language_processing&oldid=1301380737 (thanks to WayBack Machine).
- [15] Zilliz [2023], 'Everything you need to know about vector index basics', <https://zilliz.com/learn/vector-index>. [Online; accessed 14-August-2025]
In event of dead-link, or changed content, visit: <https://web.archive.org/web/20250814174522/https://zilliz.com/learn/vector-index> (thanks to WayBack Machine).